

C# 프로그래밍

동명대학교 게임학부 · 2026년 1학기

Lecture 10 Appendix

키보드 입력 · LogPanel · LINQ · delegate/event

본 강의의 6개 Lab을 위한 보조 도구와 개념 보강

강영민

동명대학교 게임학부

Contents

1	이 부록의 목적	2
2	키보드 입력 (Old Input Manager)	2
2.1	한 줄로 요약	2
2.2	세 가지 키 검사 메서드	2
2.3	KeyCode: 어떤 키를 가리킬까	3
2.4	왜 Update에 넣는가?	3
2.5	여러 키를 한 번에	3
2.6	축(Axis) 입력 — 부드러운 이동	4
2.7	Active Input Handling 설정	4
3	LogPanel: 화면에 로그 띄우기	5
3.1	Debug.Log만으로는 부족할 때	5
3.2	LogPanel.cs 전체 코드	5
3.3	static Instance: 싱글톤 패턴	6
3.4	Unity 씬 설정 (단계별)	6
3.5	Lab 코드에서 사용하기	7
3.6	안전한 호출: null 검사	8
3.7	버퍼 동작: 화면이 넘치지 않게	8
3.8	변형: 화면 출력만, 또는 Console만	9
3.9	TMP_Text가 보이지 않을 때 체크리스트	9
4	LINQ: 컬렉션을 “질의”하는 한 줄들	10
4.1	한 줄 정의	10
4.2	왜 만들어졌는가	10
4.3	두 가지 문법	10
4.4	핵심 메서드	11
4.5	실제 예 — Lab 5의 점수 통계 확장	11
4.6	작동 원리: 세 가지 C# 기능의 합	12
4.7	어디에서 쓰이는가	13
4.8	lec10에서의 위치	13
4.9	핵심 한 문장	14
5	delegate, Action, event — 구독자 패턴 깊이 보기	14
5.1	출발점: 어떤 문제를 해결하는가	14
5.2	주연 셋	14
5.3	1단계 — delegate: “메서드를 변수처럼”	15
5.4	2단계 — event: delegate 위의 안전 장치	15
5.5	3단계 — 발신자 코드 한 화면	16
5.6	4단계 — 수신자 코드의 “OnEnable/OnDisable 짝”	16
5.7	5단계 — ColorChanger도 똑같은 패턴	17
5.8	6단계 — 실행 흐름	18
5.9	7단계 — 이 설계가 “이긴다”는 증거	18
5.10	8단계 — “=> 의 세 얼굴”이 모이는 곳	19
5.11	9단계 — 본문에 던질 한 문장	19
5.12	다음 강의로 이어 줄 자연스러운 다리	19

6	종합: Lab 1을 LogPanel로 바꾸기	20
6.1	목표	20
6.2	변경할 코드	20
6.3	체크리스트	21
7	한눈에 정리	21

1. 이 부록의 목적

Lecture 10의 6개 Lab은 **공통 도구 둘**에 의존하며, Lab 5에서는 **LINQ**라는 큰 주제를 살짝 맛보고 지나간다.

- `Input.GetKeyDown(KeyCode.X)` — **키보드 입력**으로 동작을 시작.
- `Debug.Log(...)` 또는 `LogPanel.Instance.Print(...)` — **결과 출력**.
- `scores.Where(s => s >= 80).Count()` 같은 **LINQ 한 줄로** 컬렉션을 질의.
- `event Action<int> OnChanged` **구독자 패턴(Observer)**으로 “소문”을 낸다.

본 강의는 이들을 “이미 있는 것”처럼 사용하지만, 처음 만나는 학생에게는 **설정과 동작 원리**가 따로 정리될 필요가 있다. 이 부록이 그 역할을 한다.

2. 키보드 입력 (Old Input Manager)

2.1. 한 줄로 요약

```
1 if (Input.GetKeyDown(KeyCode.Space))
2 {
3     Debug.Log("Space pressed!");
4 }
```

이 한 줄을 `Update()` 안에 넣으면, 스페이스 키를 누른 그 순간 **한 번** 메시지가 출력된다.

2.2. 세 가지 키 검사 메서드

`GetKeyDown` / `GetKey` / `GetKeyUp`

세 메서드는 “언제 `true`를 돌려주느냐”만 다르다.

- `Input.GetKeyDown(K)` — 키가 **눌리는 순간 한 프레임만** `true`. “명령을 한 번만 실행”에 적합 (예: 점프, 메뉴 열기, Lab1의 색 바꾸기).
- `Input.GetKey(K)` — 키가 **눌려 있는 동안 매 프레임** `true`. “계속 작용하는 동작”에 적합 (예: 이동, 발사 연사).
- `Input.GetKeyUp(K)` — 키에서 **손을 떼는 순간 한 프레임만** `true`. “끝나는 시점에만 처리”할 때 (예: 충전 후 발사).

```
1 void Update()
2 {
3     // Once per press
4     if (Input.GetKeyDown(KeyCode.Space)) Jump();
5
6     // Every frame while held
7     if (Input.GetKey(KeyCode.RightArrow)) MoveRight();
8
9     // Once per release
```

```

10     if (Input.GetKeyUp(KeyCode.F)) ReleaseCharge();
11 }

```

2.3. KeyCode: 어떤 키를 가리킬까

KeyCode는 enum이다 (Lab 1의 Mood와 같은 종류). 키보드의 모든 키에 이름이 붙어 있다.

자주 쓰는 KeyCode 값

- 알파벳: KeyCode.A, KeyCode.B, ..., KeyCode.Z
- 숫자 (키보드 상단): KeyCode.Alpha0 ... KeyCode.Alpha9
- 숫자 (Numpad): KeyCode.Keypad0 ... KeyCode.Keypad9
- 화살표: KeyCode.UpArrow, KeyCode.DownArrow, KeyCode.LeftArrow, KeyCode.RightArrow
- 특수 키: KeyCode.Space, KeyCode.Return (=Enter), KeyCode.Escape, KeyCode.Tab, KeyCode.Backspace
- 보조 키: KeyCode.LeftShift, KeyCode.LeftControl, KeyCode.LeftAlt
- 기능 키: KeyCode.F1 ... KeyCode.F12

KeyCode.Alpha1 vs KeyCode.Keypad1

키보드 상단의 1과 우측 숫자 패드의 1은 서로 다른 키다. Lab 1에서 “키 1을 누르세요”라고 할 때 Alpha1은 상단의 1만 인식한다. Numpad에 익숙한 학생은 “눌렀는데 안 돼요”라고 할 수 있다 — 노트북에는 Numpad가 없는 경우도 많으니 기본은 Alpha1을 쓴다.

2.4. 왜 Update에 넣는가?

프레임마다 키 상태를 묻는다

Input.GetKeyDown(...)은 질문이지 이벤트가 아니다. “방금 이 키가 눌렸나요?”라고 우리가 매 프레임 묻는 구조다. 따라서 Update() (매 프레임 호출되는 메서드) 안에 넣어야 한다. Start()에 한 번만 넣으면 게임 시작 직후에 단 한 번 묻고 끝나서, 그 뒤 누른 키는 영영 감지되지 않는다.

왜 이런 구조인가? 이벤트 기반(“키가 눌리면 이 함수가 자동 호출됨”)도 가능하지만, Unity의 게임 루프는 “프레임 단위로 모든 것을 처리”한다. 입력 또한 그 흐름에 맞춰 “프레임마다 상태를 묻는” 방식이 자연스럽다.

2.5. 여러 키를 한 번에

```

1 void Update()
2 {

```

```

3     if (Input.GetKeyDown(KeyCode.Alpha1)) SetColor(Color.red);
4     if (Input.GetKeyDown(KeyCode.Alpha2)) SetColor(Color.green)
5         ;
6     if (Input.GetKeyDown(KeyCode.Alpha3)) SetColor(Color.blue);
    }

```

각 if는 독립적이다. 같은 프레임에 두 키가 동시에 눌리면 두 if가 모두 true. 보통은 한 프레임에 한 키만 눌리지만, 빠르게 입력하면 동시 처리가 일어날 수 있다는 점을 기억하자.

2.6. 축(Axis) 입력 — 부드러운 이동

Input.GetAxis: -1 ~ +1 사이의 실수

화살표/WASD로 좌우 이동을 만들 때 GetKeyDown을 쓰면 “한 번 눌릴 때마다 한 칸씩” 계단처럼 움직인다. 부드럽게 이동시키려면 Input.GetAxis(...)가 더 적합하다.

```

1 void Update()
2 {
3     float h = Input.GetAxis("Horizontal"); // -1.0 ~ +1.0
4     float v = Input.GetAxis("Vertical");
5
6     transform.position += new Vector3(h, 0, v) * Time.
7         deltaTime * 5f;
8 }

```

"Horizontal"은 기본적으로 A/D 또는 좌/우 화살표에 매핑되어 있다. "Vertical"은 W/S 또는 상/하 화살표. 이 매핑은 Edit → Project Settings → Input Manager에서 확인/수정 가능.

2.7. Active Input Handling 설정

“키를 눌러도 반응이 없어요”라면 이걸 먼저 확인

Unity 2019.x부터 새 Input System 패키지가 추가되었다. 프로젝트 설정에 따라 우리가 쓰는 옛 Input.GetKeyDown이 동작하지 않을 수 있다.

확인 경로 Edit → Project Settings → Player → Other Settings → Active Input Handling

세 가지 옵션

- Input Manager (Old) — 우리 강의가 사용하는 방식. **이게 기본 추천.**
- Input System Package (New) — 새 방식 **전용**. 옛 Input.GetKeyDown이 동작하지 않는다.
- Both — 둘 다 동작. **가장 안전한 선택.**

설정 변경 후 “Editor를 재시작해야 합니다”는 메시지가 뜬다. Apply 누르면 자동 재시작. 재시작이 끝나면 Console의 다음 경고가 사라진다:

InvalidOperationException: You are trying to read Input using the UnityEngine.Input class, but you have switched active Input handling to Input System package in Player Settings.

“프로젝트 준비” 단계에 한 줄 추가

새 프로젝트를 만들 때마다 이 설정을 한 번만 확인해 두면 평생 문제없다. Lecture 10의 “프로젝트 준비” 단계 마지막에 다음을 추가하기를 권한다:

“Edit → Project Settings → Player → Active Input Handling
을 Both로 둔다.”

3. LogPanel: 화면에 로그 띄우기

3.1. Debug.Log만으로는 부족할 때

Debug.Log("HP = 5")는 Console 창에만 출력된다. 이는 개발자에게는 충분하지만, 다음 상황에서는 불편하다:

- 발표/시연 때 Console 창을 띄워 두기 번거로울 때.
- 학생이 눈으로 직접 확인하며 “아, 이게 보이는 결과구나”라고 체감할 때.
- 빌드된 게임(.exe)에서는 Console이 없으므로 화면에 띄울 수밖에 없을 때.

LogPanel은 “Debug.Log와 같은 인터페이스로 쓰되, 화면에도 보여 주는 도우미”다.

3.2. LogPanel.cs 전체 코드

```

1 using TMPro;
2 using UnityEngine;
3
4 public class LogPanel : MonoBehaviour
5 {
6     public static LogPanel Instance { get; private set; }
7     [SerializeField] private TMP_Text text;
8
9     private string buffer = "";
10
11     void Awake() { Instance = this; }
12
13     public void Print(string msg)
14     {
15         Debug.Log(msg); // Console
16         buffer = msg + "\n" + buffer; // newest on top
17         if (buffer.Length > 800)
18             buffer = buffer.Substring(0, 800);
19     }
20 }
```

```

19         if (text != null) text.text = buffer;
20     }
21 }

```

핵심 두 가지:

- Print은 Debug.Log도 호출한다 — Console과 화면에 **동시에** 출력.
- static Instance로 어디서든 LogPanel.Instance.Print(...) 호출 가능.

3.3. static Instance: 싱글톤 패턴

“썬에 하나뿐인 객체”에 전역으로 접근하기

LogPanel은 썬에 한 개만 존재하면 충분하다 (화면 패널은 하나뿐). 이 “단 하나의 인스턴스”에 어디서든 접근하고 싶을 때 쓰는 흔한 기법이 **싱글톤(Singleton) 패턴**이다.

```

1 public class LogPanel : MonoBehaviour
2 {
3     public static LogPanel Instance { get; private set; }
4     void Awake() { Instance = this; }
5 }

```

- static 필드는 “클래스 자체에 속한 변수”. 인스턴스가 아니라 타입을 통해 접근 (LogPanel.Instance).
- Awake()는 Start()보다 먼저 한 번 호출되는 생명 주기 메서드.
- “Instance = this;” — “내가 그 유일한 인스턴스다”라고 등록.

이제 어떤 스크립트에서든 LogPanel.Instance.Print("hello")가 가능하다. “발신자가 수신자를 알 필요 없이” 호출할 수 있는 게 핵심.

3.4. Unity 썬 설정 (단계별)

Unity 구현 가이드 (단계별)

1. Canvas 만들기

Hierarchy 우클릭 → UI → Canvas. 같이 EventSystem이 자동 생성된다 (지우지 말 것).

2. TMP_Text 만들기

Canvas 우클릭 → UI → Text - TextMeshPro.

처음이라면 “TMP Essentials 임포트하시겠습니까?” 다이얼로그가 뜬다.

→ Import TMP Essentials 버튼 클릭. Examples는 임포트하지 않아도 된다.

3. TMP_Text 위치/크기 조정

새로 생긴 Text (TMP) 선택 → Inspector의 Rect Transform에서:

- Anchor를 좌상단(top-left) 으로 설정 (Anchor Presets에서 클릭).

- Width = 600, Height = 400 정도로.
- Font Size = 24, Color = 흰색 또는 노란색.

4. LogPanel.cs 부착

Canvas GameObject(또는 새 빈 객체)에 LogPanel.cs를 드래그해서 컴포넌트로 부착.

5. Inspector에서 Text 슬롯 연결 (가장 흔히 빠뜨리는 단계)

LogPanel 컴포넌트의 Text 슬롯이 None (TMP_Text)로 비어 있다. Text (TMP)를 드래그해서 그 슬롯에 떨어뜨린다.

6. 재생

Lab 코드에서 LogPanel.Instance.Print("hello")을 호출하면 화면에도 hello가 표시되고, Console에도 똑같이 hello가 찍힌다.

GameObject 구조를 그림으로

Hierarchy:

Main Camera	
Directional Light	
EventSystem	<-- Canvas 만들 때 자동 생성
Canvas	<-- LogPanel.cs (싱글톤 인스턴스)
Text (TMP)	<-- LogPanel의 Text 슬롯에 연결
Lab1	<-- Lab1_Enum.cs (실제 동작)
MoodCube	<-- Renderer

3.5. Lab 코드에서 사용하기

변경 전 (Debug.Log만 사용)

```
1 void Apply(Mood m)
2 {
3     currentMood = m;
4     targetRenderer.material.color = ...;
5     Debug.Log($"Mood = {m}");
6 }
```

변경 후 (LogPanel 사용)

```
1 void Apply(Mood m)
2 {
3     currentMood = m;
4     targetRenderer.material.color = ...;
5     LogPanel.Instance.Print($"Mood = {m}");    // changed
6 }
```

호출하는 코드는 한 줄만 바뀐다. LogPanel.Print(...) 안에서 이미 Debug.Log도 호출하므로, Console과 화면 패널 양쪽에 동시에 출력된다.

3.6. 안전한 호출: null 검사

NullReferenceException을 피하려면

LogPanel.Instance는 **썬**에 LogPanel이 부착된 GameObject가 있어야 null이 아닌 값이 된다. 깜빡 잊고 부착을 안 했다면 Instance는 null이고, LogPanel.Instance.Print(...)이 NullReferenceException을 던진다.

안전한 호출 패턴 1 — null 검사

```
1 if (LogPanel.Instance != null)
2     LogPanel.Instance.Print($"Mood = {m}");
3 else
4     Debug.Log($"Mood = {m}");    // fallback to Console only
```

안전한 호출 패턴 2 — null-조건 연산자 ?.

```
1 LogPanel.Instance?.Print($"Mood = {m}");
```

“Instance가 null이 아닐 때만 Print을 호출.” 이 경우 Instance가 null이면 Console에도 출력되지 않으므로 주의.

어느 쪽이 좋은가? 강의/실습 단계에서는 패턴 1이 더 친절하다 (썬 설정을 빠뜨려도 Console에는 찍힘). 프로덕션 코드에서는 “싱글톤이 항상 존재한다”는 전제를 둔 패턴 2도 흔하다.

3.7. 버퍼 동작: 화면이 넘치지 않게

LogPanel.Print이 하는 일을 한 줄씩 보면

```
1 public void Print(string msg)
2 {
3     Debug.Log(msg);                // (1) Console
4                                     output
5     buffer = msg + "\n" + buffer;  // (2) Prepend to
6                                     buffer
7     if (buffer.Length > 800)        // (3) Truncate
8                                     if too long
9         buffer = buffer.Substring(0, 800);
10    if (text != null) text.text = buffer; // (4) Show on
11                                     screen
12 }
```

- (1) Console에도 동시에 출력 — 개발자도 행복하다.
- (2) msg + "\n" + buffer — 새 메시지가 버퍼의 맨 앞. 결과적으로 최신이 위, 옛 것이 아래로 쌓인다.
- (3) 버퍼가 800자를 넘으면 뒤를 잘라낸다 — 메모리 무한 증가 방지. 오래된 메시지가 자동으로 사라진다.

- (4) text가 null이면(슬롯 미연결) 화면 출력만 건너뛰다. 예외를 내지 않고 “조용히 패스”.

3.8. 변형: 화면 출력만, 또는 Console만

원하는 동작에 따라 Print을 쪼개도 좋다.

```

1 public void PrintScreen(string msg)    // screen only
2 {
3     buffer = msg + "\n" + buffer;
4     if (buffer.Length > 800) buffer = buffer.Substring(0, 800);
5     if (text != null) text.text = buffer;
6 }
7
8 public void PrintConsole(string msg)    // console only
9 {
10    Debug.Log(msg);
11 }
12
13 public void Print(string msg)          // both (default)
14 {
15    PrintConsole(msg);
16    PrintScreen(msg);
17 }
```

필요에 따라 한쪽만 쓰면 된다. 강의 흐름에서는 Print(...) 하나만으로 충분하다.

3.9. TMP_Text가 보이지 않을 때 체크리스트

주의

- TMP Essentials를 임포트했는가?
Window → TextMeshPro → Import TMP Essential Resources.
- Canvas의 Render Mode는 Screen Space - Overlay인가?
기본값이라 보통 그대로 두면 된다.
- Text의 Color가 검정인데 배경도 어두운가?
Inspector에서 Color를 흰색이나 노란색으로.
- Font Size가 너무 작은가?
기본 36 권장.
- Rect Transform 크기가 0인가?
Width, Height이 글자가 들어갈 만큼인지 확인.
- LogPanel의 Text 슬롯이 비어 있는가?
“가장 흔한 실수”. Hierarchy의 Text (TMP)를 슬롯에 드래그.

4. LINQ: 컬렉션을 “질의”하는 한 줄들

4.1. 한 줄 정의

LINQ는 Language INtegrated Query의 약자다. “C# 언어에 통합된 질의(query)”라는 뜻이며, SQL 같은 사고방식으로 컬렉션을 다루게 해 준다.

한 줄로 말하면

“IEnumerable<T>에 붙는 확장 메서드들의 모음”. using System.Linq;만 추가하면 List, 배열, Dictionary 등 모든 “순회 가능한” 타입에 Where, Select, Sum, OrderBy 같은 메서드가 자동으로 붙는다.

4.2. 왜 만들어졌는가

C# 1.0 시대에는 “80점 이상의 점수가 몇 개인가?”를 다음처럼 썼다.

```
1 int count = 0;
2 foreach (var s in scores)
3 {
4     if (s >= 80) count++;
5 }
```

C# 3.0 (2007)에서 LINQ가 추가된 후:

```
1 int count = scores.Where(s => s >= 80).Count();
```

또는 더 짧게:

```
1 int count = scores.Count(s => s >= 80);
```

“무엇을 원하는가”가 한눈에 보인다. for 루프를 따라가면서 “아, 80 이상인 걸 세는 거구나”라고 해석하지 않아도 된다.

4.3. 두 가지 문법

LINQ는 같은 일을 두 가지 모양으로 쓸 수 있다.

(1) 메서드 문법 (method syntax) — 더 흔히 사용

```
1 var result = scores
2     .Where(s => s >= 80)
3     .OrderByDescending(s => s)
4     .Take(3);
```

(2) 쿼리 문법 (query syntax) — SQL을 닮음

```
1 var result = from s in scores
2               where s >= 80
3               orderby s descending
4               select s;
```

둘은 “같은 코드”로 컴파일된다

쿼리 문법은 컴파일러가 메서드 문법으로 자동 변환해 준다. 따라서 성능은 동일하다. 어느 쪽을 쓸지는 취향이지만, Sum, Count, Take 같은 단순 집계는 메서드 문법이, join 같은 복잡한 결합은 쿼리 문법이 더 읽기 좋다는 평이다. 본 강의에서는 메서드 문법만 다룬다.

4.4. 핵심 메서드

Table 1: 자주 쓰는 LINQ 메서드

메서드	역할
Where(p)	조건을 만족하는 것만 거른다
Select(f)	각 원소를 변환 (map)
OrderBy / OrderByDescending	정렬
GroupBy(key)	키 기준으로 그룹화
Join	두 컬렉션을 결합
Sum / Average / Max / Min / Count	집계
First / Single / Any / All	검색/검사
Take(n) / Skip(n)	앞 n개 / 앞 n개 건너뛰기
ToList / ToArray / ToDictionary	결과를 구체 컬렉션으로

4.5. 실제 예 — Lab 5의 점수 통계 확장

```

1 List<int> scores = new List<int> { 85, 72, 91, 60, 88, 45 };
2
3 // Filtering
4 var high = scores.Where(s => s >= 80);
5
6 // Mapping (transform)
7 var doubled = scores.Select(s => s * 2);
8
9 // Sorting
10 var sorted = scores.OrderByDescending(s => s);
11
12 // Top 3
13 var top3 = scores.OrderByDescending(s => s).Take(3);
14
15 // Aggregates
16 int sum = scores.Sum();           // 441
17 double avg = scores.Average();    // 73.5
18 int max = scores.Max();           // 91
19 int countHigh = scores.Count(s => s >= 80); // 3
20
21 // Existence check
22 bool anyHigh = scores.Any(s => s >= 80); // true
23 bool allHigh = scores.All(s => s >= 80); // false

```

4.6. 작동 원리: 세 가지 C# 기능의 합

LINQ는 마법이 아니다. 이미 있던 세 가지 언어 기능이 합쳐져 만들어졌다.

(1) 확장 메서드 (this 키워드)

```

1 namespace System.Linq
2 {
3     public static class Enumerable
4     {
5         public static IEnumerable<T> Where<T>(
6             this IEnumerable<T> source,
7             Func<T, bool> predicate)
8         {
9             // ... yield return matched items ...
10        }
11    }
12 }

```

첫 매개변수의 this 덕분에 scores.Where(...)이 마치 List의 메서드인 것처럼 호출된다. 실제로는 Enumerable.Where(scores, ...)을 부르는 셈.

using System.Linq;를 빼면 이 확장 메서드들이 “보이지 않으므로” Where가 컴파일 에러로 표시된다.

(2) 람다 식 (=>)

s => s >= 80은 “s를 받아 s >= 80을 반환하는 익명 함수”. 이는 Func<int, bool> 타입에 그대로 들어간다.

함수를 값처럼 인자로 넘기는 사고방식이 LINQ를 지탱한다.

(3) 지연 실행 (deferred execution)

```

1 var query = scores.Where(s => s >= 80);    // not executed yet
2
3 foreach (var s in query) { ... }           // executes here, once
4 foreach (var s in query) { ... }           // executes again (
    recomputes)

```

LINQ는 “아직 실행하지 않은 약속”이다

Where는 즉시 결과 리스트를 만들지 않는다. “요청이 올 때마다 한 원소씩 흘러 보내는 약속”만 만들어 둔다.

ToList(), Count(), foreach 같은 종료 메서드가 호출될 때 비로소 실제 순회가 일어난다.

이 덕분에 무한한 시퀀스도 LINQ로 다룰 수 있다 (Lab 6의 CountUp처럼). infinite.Take(5)는 “무한히 만들지만 5개만 꺼낸다”는 의미가 말 그대로 통한다.

4.7. 어디에서 쓰이는가

LINQ는 List/배열만이 아니라 다양한 “질의 가능한 소스”에 통한다.

- LINQ to Objects — 메모리 안의 List, 배열 (가장 흔함; Lab 5에서 다룬 것).
- LINQ to XML (System.Xml.Linq) — XML 트리를 질의.
- LINQ to SQL / Entity Framework — 데이터베이스 테이블을 질의.
db.Students.Where(s => s.Score >= 80)이 자동으로 SQL로 변환되어 DB로 전송 됨.
- LINQ to JSON (Newtonsoft.Json 등) — JSON 문서를 질의.
- Parallel LINQ (PLINQ) — .AsParallel() 한 줄로 멀티스레드 처리.

같은 문법으로 전혀 다른 데이터 소스를 다룰 수 있다는 점이 “Language Integrated”라는 이름값이다.

4.8. lec10에서의 위치

Lab 5에서 살짝 맛본 정도 — Sum, Average, Where(...).Count()까지.

LINQ는 그 자체로 한 강의 분량이라, 본 강의에서는 “이런 게 있다”만 보여 주고 넘어 갔다. 정렬(OrderBy), 변환(Select), 그룹화(GroupBy), 데이터베이스 연동(EF Core)까지 들어가면 별도의 한 학기 주제가 된다.

실험해 보기 — Lab 5에 LINQ 한 줄씩 추가

- Lab5_Generic.cs에 Input.GetKeyDown(KeyCode.T)로 상위 3개를 출력하는 분기를 추가.

```
1 var top3 = scores.OrderByDescending(x => x).Take(3);
2 foreach (var s in top3) Debug.Log(s);
```

- Input.GetKeyDown(KeyCode.G)로 “80점 이상은 'A', 그렇지 않으면 'F'”로 변환하여 출력.

```
1 var grades = scores.Select(x => x >= 80 ? "A" : "F");
2 foreach (var g in grades) Debug.Log(g);
```

- Input.GetKeyDown(KeyCode.B)로 “80 이상이 하나라도 있나?”와 “모두 80 이상인가?”를 출력.

```
1 Debug.Log($"Any high? {scores.Any(x => x >= 80)}");
2 Debug.Log($"All high? {scores.All(x => x >= 80)}");
```

4.9. 핵심 한 문장

참고

LINQ = “컬렉션을 SQL처럼 질의” + “람다로 조건/변환을 즉석 작성” + “확장 메서드로 자연스러운 점 표기”.

이 셋이 합쳐진 C# 3.0의 대표 기능이며, “무엇을 원하는가”에 집중한 선언적 코드를 가능하게 한다.

5. delegate, Action, event — 구독자 패턴 깊이 보기

Lab 4의 Lab4.Counter는 강의 본문에서 “event 한 줄”로 끝났지만, 그 한 줄 뒤에는 C#의 함수형 기능들이 모여 만든 작은 우주가 있다. 이 부록은 Lab 4를 처음 만나는 학생을 위한 풀이 노트다.

5.1. 출발점: 어떤 문제를 해결하는가

“카운터 값이 1 증가했다”는 사건이 일어났을 때, 여러 곳이 그 사실을 알아야 한다.

- 화면 로그가 갱신되어야 한다.
- 큐브 색이 바뀌어야 한다.
- (있다면) 사운드가 재생되고, 진동이 일어나고, UI 텍스트가 갱신된다...

Counter가 이들을 직접 호출하면, 새 반응이 생길 때마다 Counter 코드를 고쳐야 한다. 이것이 “강한 결합(tight coupling)”이다. 게임이 커질수록 Counter는 “모든 것을 아는 신”이 되고, 한 줄 추가에도 무엇이 깨질지 모르는 코드가 된다.

Lab 4의 답: Counter는 “소문”만 낸다. 듣고 싶은 사람은 알아서 구독한다. Counter는 누가 듣는지 모른다.

5.2. 주연 셋

Table 2: Lab 4의 세 컴포넌트

역할	클래스	일
발신자(publisher)	Lab4.Counter	키 누름 → 값 증가 → “소문”만 낸다
수신자 1(subscriber)	Lab4.Logger	로그를 찍는다
수신자 2(subscriber)	Lab4.ColorChanger	큐브 색을 바꾼다

화살표는 한 방향

수신자들은 발신자를 안다 (source 슬롯에 끌어다 놓았으니까). 하지만 발신자는 수신자를 모른다. 화살표가 한 방향이라는 점이 이 패턴의 본질이다.

5.3. 1단계 — delegate: “메서드를 변수처럼”

C#에서 함수를 값처럼 다룰 수 있게 해 주는 것이 delegate이다.

```
1 // "type of a function that takes int and returns void"
2 public delegate void IntHandler(int x);
3
4 IntHandler h = SomeMethod; // store a method in a variable
5 h(42); // call the method through the
        variable
```

C# 1.0에는 직접 이런 식으로 정의했다. 매번 새 delegate 타입을 만드는 게 번거로워서, .NET이 자주 쓰는 모양을 미리 만들어 두었다. 그게 Action과 Func다.

Action과 Func — .NET이 미리 만든 단축

- Action — 매개변수 없음, 반환 void
- Action<int> — int 받음, 반환 void
- Action<int, string> — int와 string 받음, 반환 void
- Func<int, bool> — int 받음, bool 반환
- Func<int, int, int> — 마지막이 반환, 앞은 매개변수

학생에게 가르칠 한 줄:

“Action<T>는 “T 받고 void 돌려주는 함수의 타입”을 위해 .NET이 미리 만든 단축이다.”

5.4. 2단계 — event: delegate 위의 안전 장치

만약 Counter에 그냥 public Action<int> OnChanged;라고 쓰면 무엇이 문제일까?

```
1 // If event keyword is missing...
2 counter.OnChanged = null; // wipe everyone's
        subscription!
3 counter.OnChanged.Invoke(99); // fake the event from
        outside!
```

구독 명단을 통째로 날리거나, 마음대로 가짜 사건을 날려 버릴 수 있다. event 키워드를 붙이면 이런 일이 컴파일 에러로 막힌다.

```
1 public event Action<int> OnChanged; // outside can only +=
        and -=
```

event가 외부에 허락하는 것

- +=로 구독 추가 — OK
- -=로 구독 해제 — OK
- 직접 호출 counter.OnChanged(42) — 컴파일 에러
- 통째로 대입 counter.OnChanged = null — 컴파일 에러

event는 “delegate에 안전 장치를 씌운 것”이라는 한 줄이 핵심.

5.5. 3단계 — 발신자 코드 한 화면

```

1 public class Lab4_Counter : MonoBehaviour
2 {
3     public event Action<int> OnChanged;
4     private int value_ = 0;
5
6     void Update()
7     {
8         if (Input.GetKeyDown(KeyCode.Space))
9         {
10             value_++;
11             OnChanged?.Invoke(value_);
12         }
13     }
14 }

```

고작 이게 전부다. Counter는:

- 누가 듣는지 모른다.
- 몇 명이 듣는지 모른다.
- 들어 줄 사람이 없을 수도 있다 (?..Invoke가 그래서 필요).

?..Invoke: 구독자가 0명일 때를 대비한 안전 호출

event는 처음에 null이다 (아직 아무도 구독하지 않음). null인 상태에서 Invoke를 부르면 NullReferenceException이 터진다. ?. (null-conditional operator)을 쓰면 “왼쪽이 null이 아닐 때만” 호출한다.

```

1 // These two lines mean the same thing
2 OnChanged?.Invoke(value_);
3 if (OnChanged != null) OnChanged(value_);

```

왜 Invoke를 써야 하나? event는 외부 직접 호출이 막혀 있다. 대신 Invoke()라는 메서드가 자기 클래스 내부에서 모든 구독자를 차례로 부른다. ?.와 결합한 ?.Invoke(...) 패턴이 사실상 표준.

5.6. 4단계 — 수신자 코드의 “OnEnable/OnDisable 짝”

```

1 public class Lab4_Logger : MonoBehaviour
2 {
3     [SerializeField] private Lab4_Counter source;
4
5     void OnEnable() { source.OnChanged += React; } // subscribe
6     void OnDisable() { source.OnChanged -= React; } // unsubscribe

```

```

7
8     void React(int v) => Debug.Log($"Logger heard: {v}");
9 }

```

세 가지를 동시에 가르칠 수 있다.

(1) += React의 “메서드 그룹”

React는 메서드 이름이다. 호출이 아니라 메서드 자체를 가리키는 표현으로 쓰였다. 컴파일러는 OnChanged가 Action<int> 타입이고 React도 (int) => void 모양임을 보고 자동으로 호환된다고 판단한다.

람다로 써도 똑같이 동작한다:

```

1 source.OnChanged += v => Debug.Log($"Logger heard: {v}");

```

단, 람다는 -=로 해제할 수 없다 (같은 람다 객체를 다시 만들 수 없으므로). “구독했다가 해제”까지 하는 패턴에서는 이름 있는 메서드를 권한다.

(2) OnEnable/OnDisable이 짝인 이유

- OnEnable — 컴포넌트가 활성화될 때마다 호출 (Play 시작, 체크박스 켜기, SetActive(true)).
- OnDisable — 컴포넌트가 비활성화될 때마다 호출 (체크박스 끄기, SetActive(false), 객체 파괴 직전).

-=를 빠뜨리면

이미 파괴된 수신자에게 발신자가 계속 호출하려 하면:

- 메모리 누수 — 발신자가 “죽은 수신자”에 대한 참조를 쥐고 있어 GC가 수신자를 회수 못 함.
- 파괴된 객체에 호출 — MissingReferenceException 발생 가능.

OnDisable에서 -=로 빼 주면 두 문제가 동시에 사라진다.

학생에게 외우게 할 한 줄:

“+=는 OnEnable에, -=는 OnDisable에. 짝을 잘못 맞추면 객체가 죽은 뒤에도 이벤트가 그쪽을 부른다.”

(3) => Debug.Log(...) — 식 본문 메서드

void React(int v) => Debug.Log(\$"Logger heard: {v}");는 식 본문 메서드(Lab 2에서 본 =>의 첫 얼굴). 본문이 한 줄일 때 { } 대신 쓸 수 있는 단축이다.

5.7. 5단계 — ColorChanger도 똑같은 패턴

```

1 public class Lab4_ColorChanger : MonoBehaviour
2 {
3     [SerializeField] private Lab4_Counter source;
4     [SerializeField] private Renderer target;
5
6     void OnEnable() { source.OnChanged += React; }

```

```

7      void OnDisable() { source.OnChanged -= React; }
8
9      void React(int v)
10     {
11         float t = (v % 10) / 10f;
12         target.material.color = Color.Lerp(Color.black, Color.
            cyan, t);
13     }
14 }

```

구조가 **Logger**와 똑같다는 점을 학생이 “느껴야” 한다. 둘은 서로의 존재를 모르고, 발신자도 둘의 존재를 모른다. 화살표는 “수신자 → 발신자” 한 방향이다(구독을 거는 것은 수신자이므로).

5.8. 6단계 — 실행 흐름

SPACE 키 한 번 → 두 곳이 동시에 반응

```

[Player presses SPACE]
|
v
Lab4_Counter.Update()
  value_++           // value_ = 1
  OnChanged?.Invoke(1)
  |
  +-----> Lab4_Logger.React(1)
  |          => Debug.Log("Logger heard: 1")
  |
  +-----> Lab4_ColorChanger.React(1)
  |          => target.color = Lerp(black, cyan, 0.1)

```

한 번의 **Invoke**가 여러 메서드를 차례대로 부른다. C#의 event는 내부적으로 “구독 메서드 리스트”를 갖고 있고, **Invoke**는 그 리스트를 순회하며 모두 호출한다. 이 자료 구조를 “멀티캐스트 delegate”라 한다.

5.9. 7단계 — 이 설계가 “이긴다”는 증거

수업에서 학생에게 직접 시켜 볼 수 있는 실험.

실험 A — 새 구독자 추가, 발신자는 한 줄도 안 고친다

Lab4_Beeper.cs를 만들어 OnChanged에 +=만 한다. Counter는 손도 안 댔는데 Beeper도 자동으로 반응한다.

```

1 public class Lab4_Beeper : MonoBehaviour
2 {
3     [SerializeField] private Lab4_Counter source;
4     void OnEnable() { source.OnChanged += React; }
5     void OnDisable() { source.OnChanged -= React; }
6     void React(int v) => Debug.Log($"BEEP x {v}");

```

```
7 | }
```

발신자 변경 없이 새 행동 추가 = 개방-폐쇄 원칙(Open-Closed Principle).

실험 B — 한 구독자가 빠져도 나머지는 영향 없다

Lab4.ColorChanger의 체크박스를 끈다. OnDisable이 호출되어 자동으로 -=가 일어난다. 로그는 여전히 찍히고, 큐브 색만 안 바뀐다. 발신자도, Logger도 아무 영향 없다.

느슨한 결합(loose coupling) — 한 부분이 빠져도 다른 부분이 살아 있다.

5.10. 8단계 — “=> 의 세 얼굴”이 모이는 곳

학생들이 헛갈려 하는 =>의 세 가지 얼굴이 Lab 4에 모두 등장한다.

Table 3: =>의 세 얼굴이 Lab 4에 모이는 자리

등장 자리	얼굴
<code>void React(int v) => Debug.Log(...)</code>	(1) 식 본문 메서드
<code>OnChanged += v => Debug.Log(v);</code> (콘솔 예시)	(2) 람다 식
(Lab 1 복습) <code>m switch { Mood.Happy => Color.yellow, ... }</code>	(3) switch 식의 가지

같은 기호, 다른 자리, 다른 의미. Lab 4를 끝낼 때 “=> 세 얼굴 표”를 다시 짚어 주면 학생들이 “아 이거 Lab 1, 2, 5 다 거기서 봤네”하며 한꺼번에 정리된다.

5.11. 9단계 — 본문에 던질 한 문장

Lab 4를 한 줄로 표현하면

“Counter는 ‘소문’만 낸다. 누가 듣는지는 Counter의 일이 아니다.”

이 한 문장이 Lab 4의 모든 것이다. `delegate/Action<T>/event`는 C#이 그 한 문장을 “컴파일러가 강제하는 형태”로 표현하기 위해 만들어 둔 도구다. Observer 패턴이라는 이름으로 알려진 이 구조는 게임 개발에서 UI 갱신, 도전 과제 진행 알림, 입력 시스템, 사운드 트리거 등 거의 모든 곳에 등장한다.

5.12. 다음 강의로 이어 줄 자연스러운 다리

- Lab 5의 `Func<int, bool>` — Lab 4에서 `Action<int>`를 보았으니, “반환 값이 있는 버전”으로 자연스럽게 이어진다. `scores.Where(x => x >= 80)`의 `x => x >= 80`이 곧 `Func<int, bool>`.
- Lab 6의 콜백 람다 — `Tick(1f, n => Debug.Log(...))`도 결국 “메서드를 인자로 넘기는” 같은 사고방식이다.
Lab 4의 “소문”이 “여러 명이 듣는 것”이라면, Lab 6의 “콜백”은 “한 명이 듣는 것”이다. 본질은 같다 — 함수를 값처럼 다룬다.

이 흐름으로 4 → 5 → 6을 연결하면 학생이 “이게 다 한 가닥의 이야기였구나”라고 깨닫게 된다.

6. 종합: Lab 1을 LogPanel로 바꾸기

본 부록의 두 도구를 한 번에 적용해 보자.

6.1. 목표

Lab 1의 “키 1/2/3 → 큐브 색 변화” 동작은 그대로 두되, Mood = Happy 같은 **출력이 화면에도 표시되도록** 만든다.

6.2. 변경할 코드

```
1 using UnityEngine;
2
3 public enum Mood { Happy, Sad, Angry }
4
5 public class Lab1_Enum : MonoBehaviour
6 {
7     [SerializeField] private Renderer targetRenderer;
8     private Mood currentMood = Mood.Happy;
9
10    void Start() { Apply(currentMood); }
11
12    void Update()
13    {
14        if (Input.GetKeyDown(KeyCode.Alpha1)) Apply(Mood.Happy)
15        ;
16        if (Input.GetKeyDown(KeyCode.Alpha2)) Apply(Mood.Sad);
17        if (Input.GetKeyDown(KeyCode.Alpha3)) Apply(Mood.Angry)
18        ;
19    }
20
21    void Apply(Mood m)
22    {
23        currentMood = m;
24        Color color = m switch
25        {
26            Mood.Happy => Color.yellow,
27            Mood.Sad   => new Color(0.3f, 0.5f, 1f),
28            Mood.Angry => Color.red,
29            -           => Color.white
30        };
31
32        targetRenderer.material.color = color;
33
34        // Was: Debug.Log($"Mood = {m}");
35        if (LogPanel.Instance != null)
36            LogPanel.Instance.Print($"Mood = {m}");
37        else
38            Debug.Log($"Mood = {m}");
39    }
40 }
```

6.3. 체크리스트

Unity 설정 점검

1. **Active Input Handling**이 Both 또는 Old인가?
(Edit → Project Settings → Player → Other Settings)
2. **Canvas + Text(TMP) + LogPanel.cs**가 씬에 있는가?
3. **LogPanel의 Text** 슬롯에 Text (TMP)가 연결되어 있는가?
4. **Lab1_Enum의 Target Renderer** 슬롯에 MoodCube가 연결되어 있는가?
5. Play → 1, 2, 3 키.
큐브 색이 바뀌고, 화면 상단에 Mood = Happy 등이 **누적**되는가?

실험해 보기

- Lab 2 Lab2_Property의 Report()도 Debug.Log 대신 LogPanel.Instance.Print으로 바꿔 보자. “HP = 9/10”이 화면에 누적된다.
- Lab 6 Lab6_Coroutine의 콜백 람다를 `n => LogPanel.Instance.Print($"Tick #{n}")`으로 바꿔 보자. 매초 카운트가 화면 위에서 아래로 누적된다.
- LogPanel의 버퍼 한도 800을 200으로 줄이면 어떻게 되는가? 오래된 로그가 더 빨리 사라진다.

7. 한눈에 정리

Table 4: 본 부록의 핵심 도구

용도	호출 한 줄
키가 눌리는 순간 한 번 처리	<code>Input.GetKeyDown(KeyCode.X)</code>
키를 누르고 있는 동안 처리	<code>Input.GetKey(KeyCode.X)</code>
키에서 손을 떼는 순간 처리	<code>Input.GetKeyUp(KeyCode.X)</code>
축 입력 (-1 ~ +1)	<code>Input.GetAxis("Horizontal")</code>
Console에만 출력	<code>Debug.Log("...")</code>
Console과 화면 둘 다 출력	<code>LogPanel.Instance.Print("...")</code>

이 도구들만 알면

Lecture 10의 6개 Lab은 **입력**과 **출력**을 두 도구로 통일하고, Lab 4의 **event/구독자 패턴**으로 “느슨한 결합”을 익히고, Lab 5에서는 **LINQ**로 컬렉션을 한 줄에 다룬다. 본문은 **C# 개념**(enum, 프로퍼티, 인터페이스, 이벤트, 제네릭, 코루틴)에 집중하고, “입력/출력/질의/구독”은 이 부록을 한 번 익혀 두면 따로 신경 쓰지 않아도

Table 5: LINQ 한 줄들 (using System.Linq; 필요)

원하는 것	LINQ 한 줄
조건으로 거르기	<code>xs.Where(x => x >= 80)</code>
변환하기	<code>xs.Select(x => x * 2)</code>
정렬	<code>xs.OrderBy(x => x)</code>
내림차순 정렬	<code>xs.OrderByDescending(x => x)</code>
앞 N개	<code>xs.Take(3)</code>
합/평균/최대/최소	<code>xs.Sum() / Average() / Max() / Min()</code>
조건을 만족하는 개수	<code>xs.Count(x => x >= 80)</code>
하나라도/모두	<code>xs.Any(p) / All(p)</code>
구체 컬렉션으로	<code>xs.ToList() / ToArray()</code>

Table 6: event와 구독자 패턴 (Lab 4)

용도	호출 한 줄
이벤트 선언 (발신자)	<code>public event Action<int> OnChanged;</code>
구독 추가 (수신자)	<code>source.OnChanged += React;</code>
구독 해제 (수신자)	<code>source.OnChanged -= React;</code>
안전 호출 (발신자)	<code>OnChanged?.Invoke(value);</code>
구독 시점 / 해제 시점	<code>OnEnable / OnDisable</code> 짝

된다.